DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOPS

TP de synthèse 2
Projet « Remboursement »

De la donnée versionnée au modèle expliqué

Étudiant	Matricule	Graine G
AiT HAMiDA Saad	R-2023-000064	64

Ce compte rendu suit les questions A1 à F13. Les noms de fichiers entre crochets renvoient aux preuves du projet livré. Les données brutes et préparées sont suivies par DVC.

Phase A — Projet, Git et données

A1 — Exclusions Git

`git check-ignore -v -- .venv/ mlruns/` affiche la règle de `.gitignore` qui couvre chacun des deux répertoires. `git ls-files -- .venv mlruns` ne renvoie aucun fichier suivi. Ces commandes contrôlent les ajouts Git ordinaires. Un ajout forcé contournerait `.gitignore`.

Preuve dans la sortie Git [a1-git-check-ignore.txt].

A2 — Provenance du classeur

Le fichier XLSX téléchargé depuis le portail officiel mesure **530 243 octets**. Son MD5 est `68a57c72d5eeb4c5256fba3322393d32`. Le CNOPS publie ce jeu sous **Open Data Commons Open Database License (ODbL)**. La fiche indique le **13 décembre 2021 à 16 h 26 UTC** pour la dernière modification du jeu. Le portail date séparément la modification de la ressource XLSX du **18 décembre 2017 à 20 h 26 UTC**.

Preuves dans les métadonnées du portail [a2-portail.json] et la sortie de l'audit [a2-a5-mesures.txt].

A3 — Dimensions et cible

Le classeur comporte **5 917 présentations** et **12 colonnes**, hors en-tête. Parmi elles, **1 977** ont un taux de remboursement de `0 %`, soit **33,41 %**.

A4 — Dosages

`DOSAGE1` compte **181 cellules vides** et **946 valeurs non convertibles en un seul nombre**, cellules vides comprises. `1 / 125` en `G / MG` représente des doses de 1 g et 125 mg. `100 / 12.5` et `500 / 62.5`, en `MG / MG`, donnent chacun les doses en mg de deux composants. Les remplacer par zéro perd la quantité de chaque composant et confond les dosages composés avec les dosages absents ou réellement nuls dans la variable préparée.

A5 — Présentations par DCI

AMOXICILLINE est la DCI la plus fréquente avec **157 présentations**. Le nombre de présentations d'une même DCI peut refléter la variété de dosages, de formes ou de produits disponibles pour une molécule. Cette information pourrait être liée à sa couverture et doit être évaluée sur le jeu de test.

A6 — Découpage stratifié

La graine G, obtenue à partir des quatre derniers chiffres du matricule, vaut 64. La stratification conserve la proportion globale de classe 1 dans le jeu de test, à l'arrondi près. Le médicament est sélectionné au rang G modulo 1977 dans le classeur brut, parmi les présentations non remboursées.

A6 Entrainement : 4437 | Test : 1480

A6 Classe 1 dans le test : 495 (33.45%)

A6 Variables : ppv, ph, prix_br, ecart_br, dosage, generique, nb_meme_dci, injectable, comprime

Medicament personnel, rang 64 : SOLIAN 100 MG/ML (CODE 6118010120312)

Phase B — Versionnement et pipeline

B1 — Classeur suivi par DVC

Git suit le pointeur du classeur, le fichier d'exclusion du répertoire et la configuration DVC. Le pointeur contient le MD5 **68a57c72d5eeb4c5256fba3322393d32**, identique à A2. Sortie et pointeur [b1-suivi-dvc.txt]

B2 — Stockage local

Le stockage `../stockage-dvc` contient des objets sous `files/md5/`. Les deux premiers caractères de l'empreinte servent de sous-répertoire. DVC retrouve le nom logique à partir du pointeur Git et stocke le contenu par empreinte pour dédupliquer les objets. Configuration et poussée [b2-stockage-dvc.txt]

B3 — Deux versions et restauration

La V2 conserve **5 709 présentations** dans l'ordre du classeur, après retrait des **208** lignes à PPV nul. Les tags sont `donnees-v1` et `donnees-v2`. La V2 est un classeur dérivé et le fichier V1 d'origine est conservé à son chemin initial. Pour prouver la restauration, la copie de travail a été mise à l'écart, puis `git switch --detach donnees-v1` et `dvc pull data/brut/ref-des-medicaments-cnops-2014.xlsx.dvc` ont été exécutés. Les empreintes SHA-256 sont identiques. Preuve de restauration [b3-versions.txt]

B4 — Graphe et responsabilités

Étape	Dépendances	Paramètres	Sorties
Préparer	Classeur et scripts de préparation	Source, version, graine, part de test, chemins CSV	Jeux d'entraînement et de test
Entraîner	CSV d'entraînement et scripts de la forêt	Graine, arbres, profondeur, chemins	Modèle sérialisé
Évaluer	CSV de test, modèle et scripts de mesure	Seuil F1, chemins	F1, précision et rappel de classe 1

Graphe DVC [dvc-dag.txt] · Configuration des étapes [../dvc.yaml]

B5 — Invalidation du cache

Le second `dvc repro` signale que le pipeline est à jour. Passer de 12 à 16 relance l'entraînement et l'évaluation. Le classeur, la graine et la part de test restent identiques, donc la préparation conserve ses sorties. Le retour à 12 réutilise les résultats en cache. Sorties de reproduction [b5-repro.txt]

B6 — Expériences

Profondeur	F1 classe 1
8	0,696855
12	0,768683
16	0,783217

La profondeur **16** donne le meilleur F1 mesuré sur ce test, avec environ 0,0145 de mieux que 12. Elle est retenue dans la comparaison B6. Les expériences temporaires laissent les valeurs de `params.yaml` identiques au commit de référence. Le modèle de référence des phases suivantes garde la profondeur **12** imposée. Résultats détaillés [b6-experiences.csv]

B7 — Validation

La préparation vérifie la présence des 12 colonnes, la validité des prix (finis et non négatifs) et les valeurs de remboursement autorisées (0 % ou 70 %). Elle contrôle aussi la présence de lignes et les statuts P/G. Sur la copie privée de la colonne PPV, le validateur renvoie le code 1 et le message « Validation refusee : colonnes manquantes : PPV ». Sortie du validateur [b7-validation.txt]

B8 — Profondeur 2

L'expérience forcée réexécute validation, préparation, entraînement et évaluation. Le **F1 de classe 1 vaut 0,344948**, sous le seuil **0,65** lu dans `qualite.f1_min` de `params.yaml`. DVC conserve l'évaluation pour la preuve et le modèle est déclaré non admissible au registre. Journal du pipeline [b8-profondeur-2.txt]

Vérification après la phase C

Une réextraction Git sous Windows avait changé la fin de ligne de valider.py sans changer son contenu. Les règles de .gitattributes fixent la fin de ligne LF des scripts et fichiers YAML, de sorte que DVC conserve les mêmes empreintes pour un même code. Deux reproductions après normalisation [b5-portabilite.txt]

Phase C — Suivi, registre et service

C1 — Exécutions sur les données V2

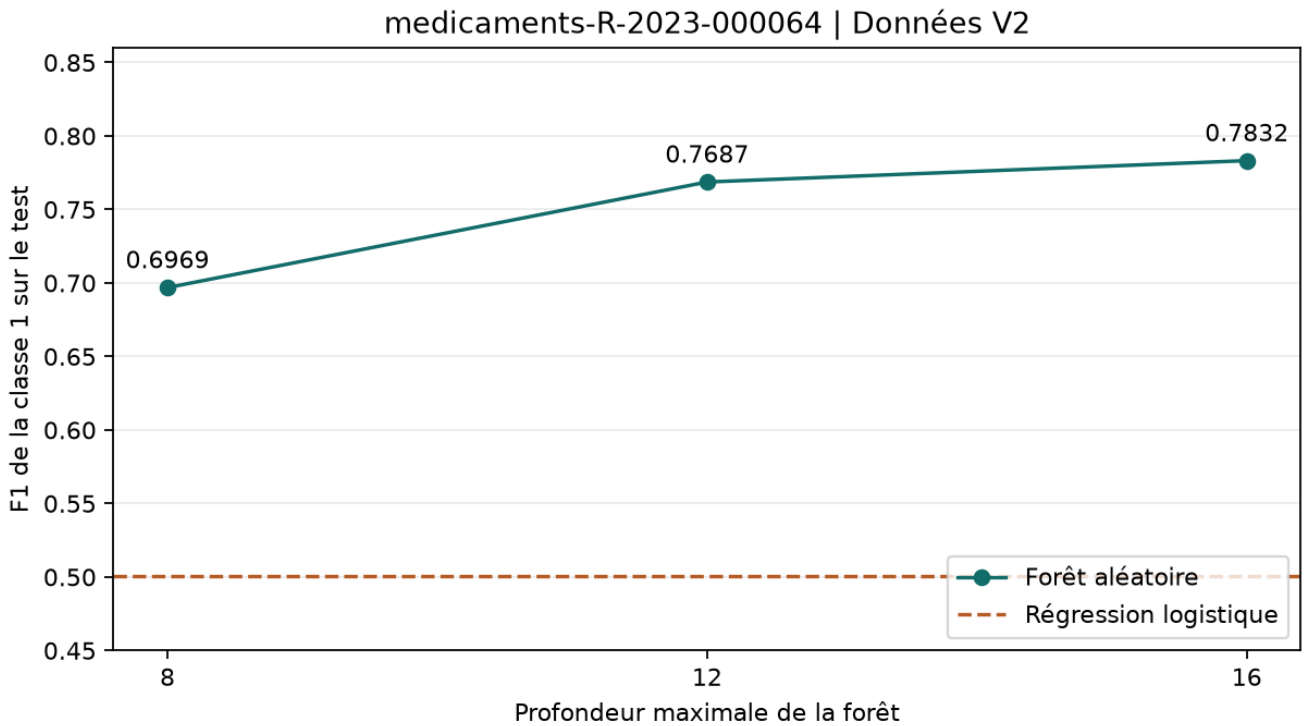
Modèle	F1 classe 1	Précision	Rappel
Forêt, profondeur 8	0,696855	0,814706	0,608791
Forêt, profondeur 12	0,768683	0,835052	0,712088
Forêt, profondeur 16	0,783217	0,833747	0,738462
Régression logistique	0,500692	0,675373	0,397802

Les quatre exécutions réussies partagent le même jeu de test et journalisent leurs paramètres, leurs métriques et leur modèle. Le premier essai de sérialisation de la forêt a échoué. Les quatre résultats du tableau proviennent des exécutions terminées après avoir déclaré explicitement le type scikit-learn des arbres formés localement. Identifiants et métriques [c1-metriques.csv]

C2 — Comparer les réglages

Dans la vue de comparaison MLflow, le **graphique à coordonnées parallèles** peut relier le paramètre `profondeur` à la métrique `f1`. On sélectionne les trois forêts en conservant la graine, les données et les 200 arbres identiques. C'est la profondeur qui varie entre ces essais. Le F1 augmente de 0,0864 de 8 à 16, surtout parce que le rappel monte de 0,6088 à 0,7385. La régression logistique est un comparatif de famille de modèles et ne possède pas de profondeur. Guide MLflow et graphique

L'accès de l'assistant au navigateur local a été refusé. La preuve jointe est un graphique calculé à partir des quatre exécutions journalisées. Graphique F1 et profondeur [c2-f1-profondeur.png]



F1 des trois profondeurs et de la régression logistique, à partir des exécutions MLflow

C3 — Signature du modèle de référence

La signature du modèle V2 à profondeur 12 comporte **neuf entrées obligatoires**. `ppv` est de type **double** et `generique` de type **long**. Le service applique ce schéma avant la prédiction, ce qui permet de rejeter une entrée manquante ou incompatible. L'exemple d'entrée est enregistré avec le modèle. Signature et exemple issus de MLflow [c3-signature.json]

C4 — Enregistrement, alias et probabilité

La forêt de référence V1 obtient un F1 de 0.769902. La V2 de référence obtient 0,768683. Les deux dépassent 0,65. MLflow a enregistré la V1 en version 1 et la V2 en version 2. Les alias après promotion sont production → 2 et staging → 1.

Le modèle chargé par l'alias production attribue à **SOLIAN 100 MG/ML** (CODE 6118010120312) une probabilité de **0.792146** pour la classe 1. Preuve du registre [c4-promotion.json] · Exécution V1 [c4-execution-v1.json]

C5 — Retour arrière

L'alias `production` est revenu de la version 2 (données V2) à la version 1 (données V1). L'application conserve l'URI `models:/remboursement-rf@production`. Un nouveau chargement résout l'alias à nouveau. Un processus ayant déjà chargé le modèle doit le recharger ou redémarrer pour appliquer le changement. Preuve du retour arrière [c5-retour-arriere.json]

C6 — Service du modèle V1 sur le port 8000

La requête complète envoyée à `/invocations` pour le médicament personnel renvoie HTTP 200 avec `{"predictions":[1]}`. La requête privée de `dosage` renvoie HTTP 400 avec une erreur de schéma indiquant la colonne manquante. Requêtes et réponses JSON [c6-reponses-http.json]

C7 — Préparer les données brutes au moment de servir

Le service actuel demande les neuf variables déjà calculées. Une application qui ne dispose que du référentiel devrait sinon reproduire le calcul et risquer un écart avec l'entraînement. On peut servir un composant qui reçoit les colonnes brutes nécessaires et applique la même préparation de variables, sans exiger le taux de remboursement, qui sert à construire la cible. Le composant associe à chaque version du modèle les fréquences de DCI calculées sur son classeur de référence. Une DCI absente de ce référentiel reçoit un compte de **0**. Pour une DCI connue, le compte est celui du référentiel associé à la version du modèle. Préparation commune [../src/preparer.py] · Préparation de la requête personnelle [../src/medicament_personnel.py]

C8 — Exécution avec MLproject

`mlflow run .` lit le point d'entrée, valide les paramètres déclarés, lance la commande et associe l'exécution au suivi MLflow. `python_env.yaml` décrit la version de Python et les dépendances. La démonstration locale utilise l'environnement déjà installé avec `--env-manager local`. Une exécution directe lance le script avec les arguments choisis par l'utilisateur et peut aussi journaliser des résultats, grâce aux appels MLflow de ce projet. Elle ne passe pas par les paramètres et l'environnement déclarés du projet. Sortie de la commande [c8-mlflow-run.txt] · Point d'entrée [../MLproject]

Phase D — AutoML et low-code

Le classeur V1 est commun aux trois approches, car Orange part du XLSX brut. Les 4 437 lignes d'entraînement et 1 480 lignes de test proviennent du même découpage stratifié avec G. Le F1 concerne la classe 1.

D1 — FLAML à 60 secondes

FLAML sélectionne **LightGBM**. Son F1 de test est **0,889340**, contre **0,769902** pour la forêt de référence V1, soit un gain de 0,1194 sur ce test. Le budget FLAML sert à optimiser le F1 pendant la recherche. Exécution FLAML [d1-flaml.json] · Référence V1 [c4-execution-v1.json]

D2 — FLAML à 120 secondes

LightGBM reste en tête et le F1 de test passe à **0,890467**, soit **+0,001127**. Une recherche plus longue explore davantage de réglages et optimise la validation interne. Le score sur les lignes de test peut augmenter, stagner ou baisser. Exécution FLAML [d2-flaml.json]

D3 — H2O AutoML

Le meilleur modèle est **GBM_4** dans la session enregistrée. Le classement utilise l'**AUC**, qui vaut **0,930957** en tête du leaderboard. Son **F1 sur le test tenu à l'écart est 0,845996**. Les réseaux de neurones et les ensembles empilés sont exclus pour que la limite de **10 modèles** s'applique au leaderboard complet. Résultats [d3-h2o.json] · Classement H2O [d3-classement-h2o.csv]

D4 — Registre et latence

Le LightGBM FLAML de 120 s est enregistré sous `remboursement-automl@production` et sert sur **8100**. Les deux services ont reçu la même requête après un appel d'échauffement chacun, puis **50 appels séquentiels par service**. La latence HTTP médiane mesurée sur cette machine est de **22,92 ms** pour la forêt (8000) et **7,11 ms** pour LightGBM (8100). Registre [d4-registre.json] · Mesures brutes [d4-latences.json]

D5 — Workflow Orange

Le widget File lit le XLSX brut. Le workflow conserve en métas `CODE`, `NOM`, `DCI1`, `UNITE_DOSAGE1`, `PRESENTATION` et `FORME` pour identifier et interpréter les lignes. DCI et forme contribuent aux variables dérivées. Les valeurs brutes de `DOSAGE1`, des prix et de `PRINCEPS_GENERIQUE` sont remplacées par les variables prescrites. `TAUX_REMBOURSEMENT` sert uniquement à créer la cible discrète. Les scripts de branche partagent les mêmes indices stratifiés calculés avec G.

Apprenant Orange	AUC	F1 classe 1
Forêt aléatoire	0,924010	0,769902
Régression logistique	0,803811	0,530414
Naive Bayes	0,799920	0,604797

Le tableau reprend les résultats du widget Test and Score en mode test séparé et le F1 de la classe 1. Workflow Orange [../orange/remboursement.ows] · Mesures Orange [d5-orange.json] · Rejeu du Canvas [d5-canvas.json]

D6 — Faux négatifs

La forêt, meilleure par AUC et F1, manque **142 présentations non remboursées**. Sa matrice met les classes réelles en lignes et les classes prédites en colonnes.

Classe réelle	Prédite 0	Prédite 1
0	916	69
1	142	353

Matrices mesurées [d5-orange.json]

D7 — Choix pour la production

Le code avec DVC rend la forêt déterministe à graine et environnement constants. MLflow lie paramètres, données et modèles pour les deux AutoML. FLAML a le meilleur F1 observé et la plus faible latence, mais un budget en secondes dépend des ressources du poste. H2O a un budget en modèles et expose son leaderboard. Orange fournit un workflow visuel et une matrice facilement inspectable. Les fichiers, versions et paramètres d'Orange doivent être archivés pour rejouer l'expérience.

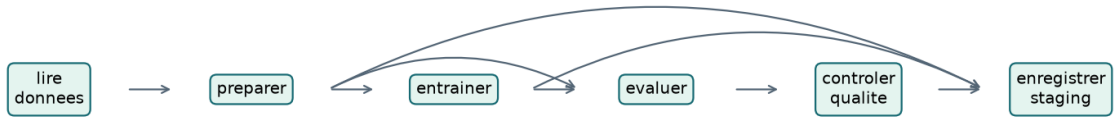
Je retiendrais **LightGBM FLAML** comme candidat d'après ces mesures, avec sa version et son environnement figés. Avant une décision réelle, je vérifierais que les entrées sont disponibles à la date de décision et j'évaluerais la qualité sur des données séparées dans le temps. Comparaison des résultats [d4-registre.json]

Phase E — Réentraînement automatisé

E1 — Graphe et états du flux

Le flux `reentrainement-medicaments` lit le classeur, prépare et sépare les données, entraîne, évalue, contrôle le F1 puis enregistre sous `staging`. Le graphe est reproduit à partir des nœuds et liens réellement retournés par l'API Prefect.

reentrainement-medicaments | graphe Prefect, exécution Completed



Graphe de la première exécution Prefect

L'exécution normale V2 se termine **Completed**, F1 **0,768683**, et place la version **3** sous `staging`. Avec une profondeur de 2, `controler-qualite` échoue avec « Modele refuse : F1 0.344948 < seuil 0.65. ». L'état final est **Failed** et l'enregistrement n'est pas atteint. Exécution normale [e1-preuve-normal.json] · Exécution refusée [e1-preuve-profondeur-2.json]

E2 — Planification mensuelle

Le déploiement `mensuel` est actif avec l'expression **`15 3 1 * *`, dans le fuseau** Africa/Casablanca**. Le serveur Prefect a créé une exécution** Scheduled le 1er décembre 2026 à 03 h 15**. Le runner se lance par `python flows/preuves_prefect.py --servir`. Déploiement et exécution planifiée [e2-planification.json]

E3 — Fichier indisponible

Le classeur V2 a été déplacé temporairement. Le journal montre **trois lectures** pour une exécution initiale et **deux réessais** espacés de deux secondes. La tâche `lire-donnees` puis le flux terminent **Failed**, avec `FileNotFoundError`. Le classeur a été remis à son chemin d'origine et son SHA-256 vérifié après l'essai. États et journal [e3-preuve.json]

E4 — Surveillance Node-RED

Les nœuds suivent cet ordre. **Inject** envoie la requête toutes les 60 s. **HTTP Request** envoie un POST à `127.0.0.1:8000/invocations`. **Switch** teste si la première prédiction vaut 1. **Change** construit l'alerte et **Debug** l'affiche dans le panneau et dans le journal. Le service a déclenché deux alertes à une minute d'intervalle pour le médicament personnel. Flux exporté [../nodered/remboursement.json] · Journal du service [e4-alerte.txt]

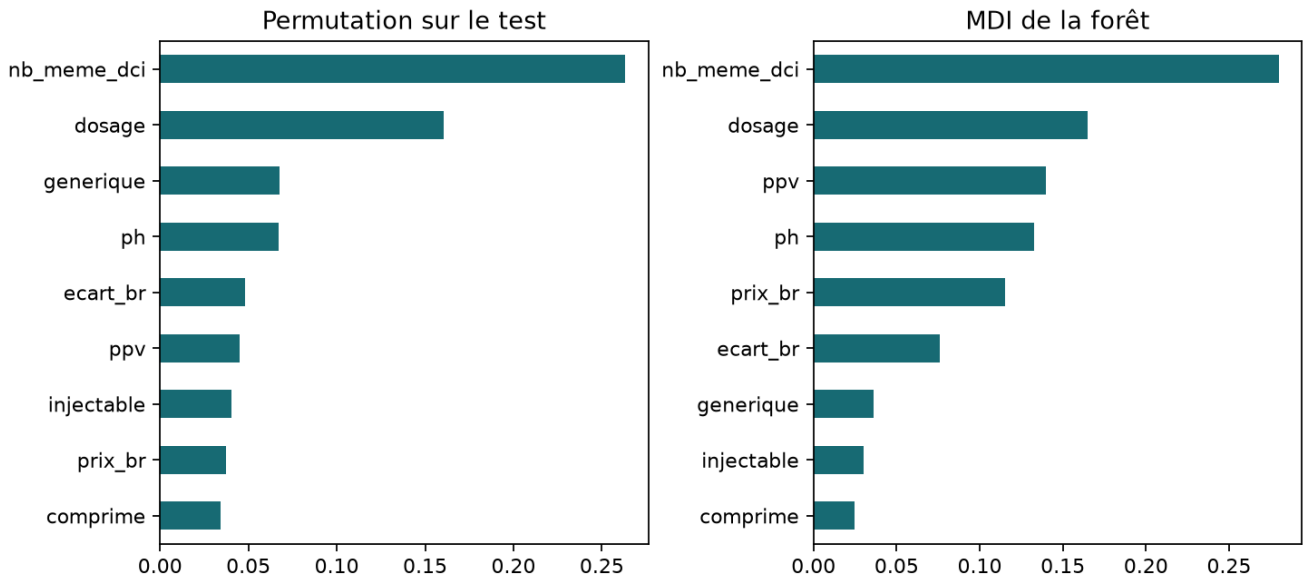
Phase F — Explicabilité et audit

Sauf indication contraire, les mesures portent sur la forêt de référence V1 chargée depuis le registre MLflow. La classe 1 désigne une présentation non remboursée.

F1 — Importance globale

Rang	Permutation sur le test, baisse du F1	MDI, réduction d'impureté
1	nb_meme_dci · 0,263217	nb_meme_dci · 0,279866
2	dosage · 0,160679	dosage · 0,165141
3	generique · 0,067422	ppv · 0,139787

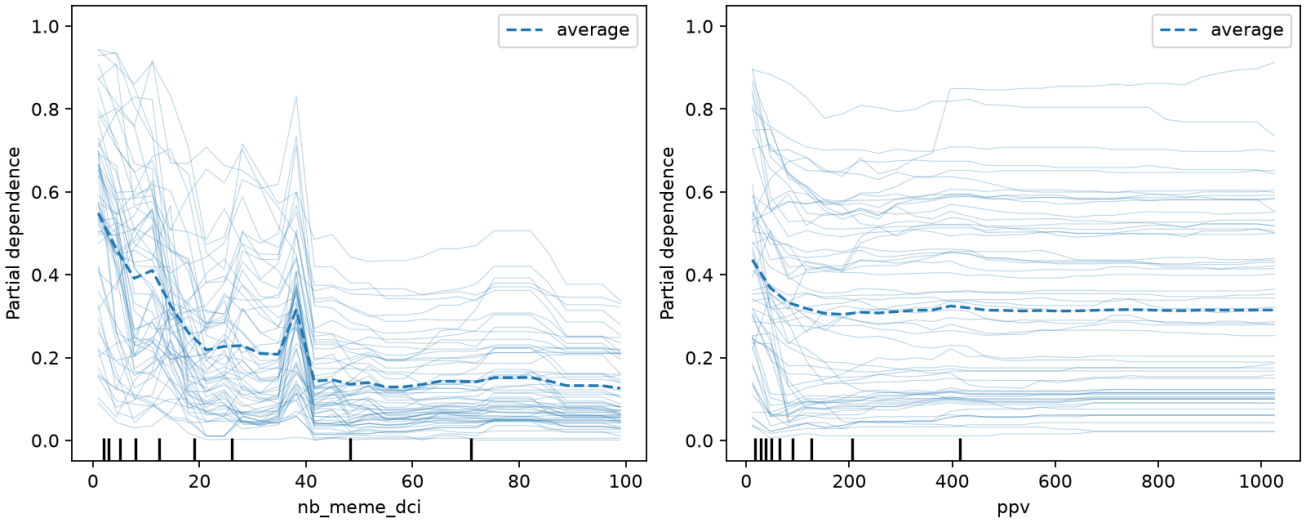
La permutation perturbe une variable à la fois, dix fois avec G, et mesure la baisse de F1 sur le test. La MDI agrège le gain d'impureté aux nœuds pendant l'entraînement et peut donner plus de poids aux variables ayant beaucoup de seuils possibles. Les prix sont liés entre eux, ce qui peut aussi réduire l'effet observé quand un seul prix est permuté. Valeurs complètes [f1-resultat.json]



Importances de la forêt V1, permutation et MDI

F2 — PDP et ICE

Sur la grille du graphique, la moyenne prédite pour `nb_meme_dci` descend globalement de **0,548 à 0,126** entre 1 et 99 présentations. Pour `ppv`, elle baisse de **0,436 à 0,315** entre environ 12 et 1 025 MAD, avec des variations locales. Les ICE ne sont pas parallèles. Entre les deux extrémités de la grille PPV, le 10e percentile des écarts individuels vaut **−0,258**, alors que le 90e vaut **+0,010**. La réponse du modèle à un prix dépend donc des autres caractéristiques de la ligne. Ces courbes décrivent le modèle et peuvent explorer des combinaisons de prix qui ne se rencontrent pas dans le référentiel. Grilles et quantiles [f2-resultat.json]



PDP et courbes individuelles ICE pour le nombre de présentations et le PPV

F3 — Médicament personnel

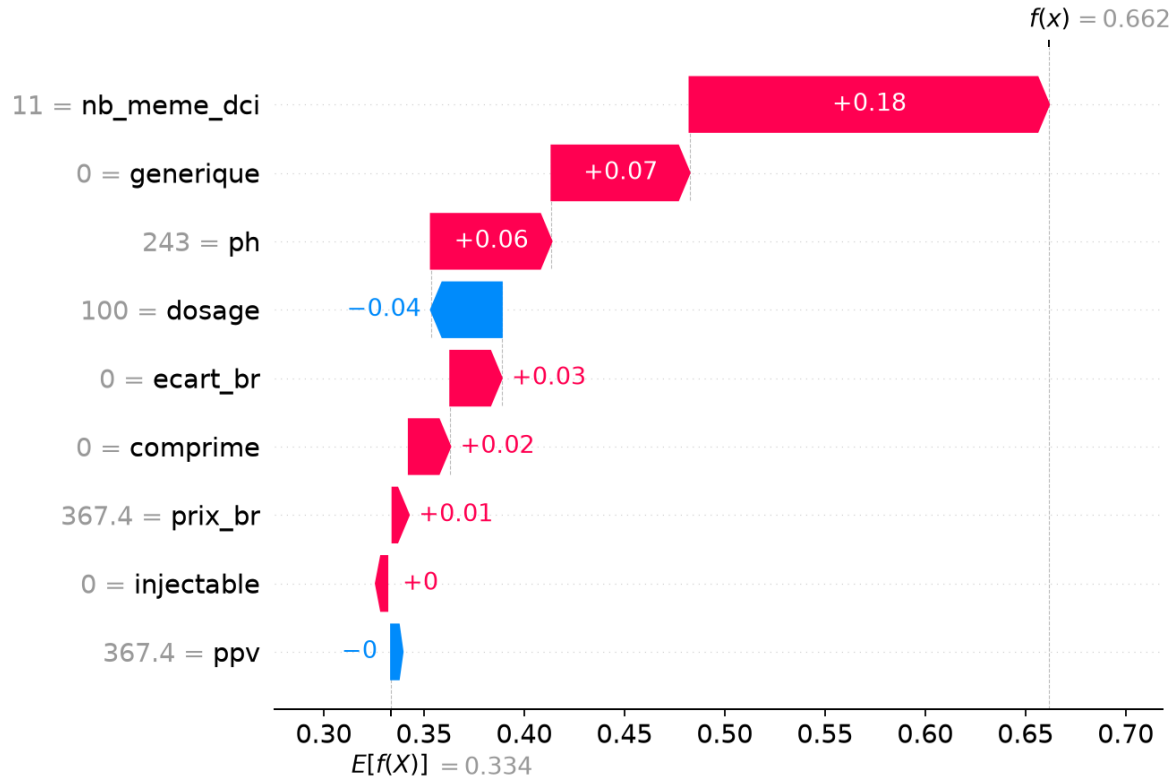
SOLIAN 100 MG/ML, DCI **AMISULPRIDE**, CODE **6118010120312**, est au rang 64 parmi les présentations non remboursées.

Variable	Valeur	Variable	Valeur
ppv	367,4	ph	243
prix_br	367,4	ecart_br	0
dosage	100	generique	0
nb_meme_dci	11	injectable	0
comprime	0		

La probabilité de classe 1 est **0,661805**. Au seuil 0,5, la forêt prédit **1**, la classe observée. Entrées et prédiction [f3-resultat.json]

F4 — Attribution SHAP

La valeur de base est **0,333632**. Les trois plus fortes contributions absolues sont **nb_meme_dci +0,179306**, **generique +0,068827** et **ph +0,060190**. La somme des neuf contributions vaut **0,328173**. La base plus cette somme vaut **0,661805**, ce qui reproduit la probabilité. Décomposition complète [f4-resultat.json]



Décomposition SHAP de la probabilité de classe 1

F5 — Message au gestionnaire

Le modèle estime à 66,2 % la probabilité que cette présentation ne soit pas remboursée. Le nombre de présentations de la même molécule, le statut princeps et le prix hospitalier sont les principaux éléments qui font monter cette estimation et justifient d'examiner le dossier.

F6 — Contrefactuels

Inverser uniquement le statut de princeps à générique ramène la probabilité de **0,661805 à 0,298520**. Modifier ce statut sans modifier le produit ne constitue pas une intervention réaliste sur une présentation existante. Un scénario plausible à étudier est une baisse administrative du PPV de **367,40 à 330,66 MAD**, à prix de base constant, avec l'écart recalculé à **−36,74 MAD**. Le modèle donne alors **0,644179**. Scénarios et scores [f6-resultat.json]

F7 — Stabilité LIME

Signe sur les cinq graines G à G+4	Variables
Toujours positif	ph, prix_br, ecart_br, nb_meme_dci, injectable, comprime
Toujours négatif	ppv, dosage
Variable	generique · −, −, +, −, +

Le voisinage tiré par LIME change entre les graines. Je retiens le sens stable des principales contributions comme recoupement et signale que le signe de `generique` est instable dans cette approximation locale. Cinq essais, poids et R^2 locaux [f7-resultat.json]

F8 — Équité par groupe

Forêt	Groupe	Prédit classe 1	Fausses alertes
Avec generique	Générique	16,53 %	3,74 %
Avec generique	Princeps	40,05 %	11,32 %
Sans generique	Générique	15,84 %	4,10 %
Sans generique	Princeps	37,00 %	10,85 %

L'écart de taux de classe 1 passe de **23,52 à 21,16 points**. L'écart de fausses alertes passe de **7,58 à 6,75 points**. Des variables corrélées au statut ou des distributions de dossiers différentes peuvent maintenir des écarts. Un taux de fausses alertes supérieur signifie davantage de dossiers princeps réellement remboursés signalés à tort, donc davantage de travail pour les gestionnaires et d'examens injustifiés pour les acteurs concernés. Mesures Fairlearn [f8-resultat.json]

F9 — Fuite potentielle par la base de remboursement

Si la prédiction a lieu avant la procédure, `PRIX_BR` n'est pas encore connu. L'utiliser à ce moment crée une fuite temporelle potentielle, également présente dans `ecart_br` qui en dérive. Retirer les deux variables fait passer le F1 de **0,769902 à 0,769231**, soit **−0,000671** sur ce test. Une version destinée à l'amont pourrait les supprimer avec un faible coût mesuré ici. Comparaison sur le même test [f9-resultat.json]

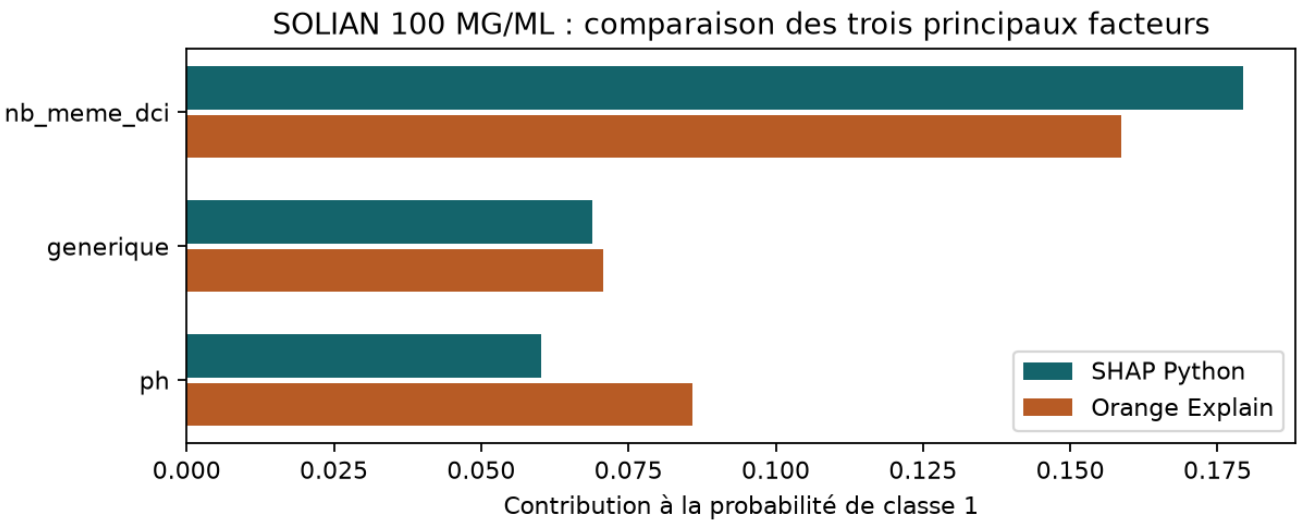
F10 — Modèles lisibles

Modèle	F1 classe 1
Forêt de référence	0,769902
Logistique standardisée	0,512438
Arbre de profondeur 2	0,389685

Le coefficient standardisé de `generique` est **−0,179903**. Le rapport de cotes est **0,8354** pour +1 écart-type et **0,6977** en passant de princeps à générique avec les autres variables fixes. L'arbre prédit 1 quand **nb_meme_dci ≤ 12,5** et **dosage ≤ 0,19**. Les autres feuilles prédisent 0. L'écart de F1 est compatible avec des interactions et des effets de seuil mieux représentés par la forêt sur ce jeu de test. Coefficient et règles complètes [f10-resultat.json]

F11 — Comparaison avec Orange Explain

Orange Explain Prediction donne **0,661805** pour la classe 1. Les deux premières contributions absolues sont **nb_meme_dci +0,158587** et **ph +0,085829**. SHAP en Python place **nb_meme_dci** en tête et **generique** en deuxième. Orange utilise un ensemble de référence issu du jeu d'entraînement, ce qui change la valeur de base (0,328908 au lieu de 0,333632). Les méthodes de traitement des variables dépendantes ou les valeurs retenues pour le fond de l'explainer peuvent aussi redistribuer les contributions entre prix et statut sans changer la prédiction. Sortie Explain [f11-orange-explain.json] · Rejeu du widget Canvas [f11-canvas.json]



Attributions pour la classe 1, Python et Orange

F12 — Journalisation et fiche modèle

Les graphiques d'importance, de PDP/ICE, de SHAP et de comparaison Orange, ainsi que `MODEL_CARD.md`, sont journalisés dans le run MLflow de la V1 de production sous `audit-explicabilite/`. La rubrique la plus délicate est la disponibilité des variables à la date de décision. Le référentiel donne le prix de base, mais il ne décrit pas le moment où cette valeur devient accessible pour chaque dossier. La fiche distingue donc les conditions de l'exercice et celles d'un usage en amont. Liste des artefacts MLflow [f12-artefacts.json] · Fiche modèle [../MODEL_CARD.md]

F13 — Refus d'un modèle avec fuite

Dans l'essai V1, `decision_cnops` copie la cible et **178 valeurs sur 5 917** sont inversées à graine G, soit 3 % à l'arrondi près. Le modèle ainsi entraîné obtient un **F1 de 0,956000** et franchit le seuil de qualité. Le contrôle d'explicabilité calcule les importances par permutation et compare le schéma du modèle à la liste des neuf entrées permises. `decision_cnops` est hors liste et arrive en tête, avec une baisse de F1 de **0,571147** après permutation.

La tâche `controler-explicabilite` et le flux terminent **Failed**. L'enregistrement est bloqué et `staging` conserve la version **3**. Une reproduction DVC du pipeline de référence redonne le F1 V2 **0,768683**. Graphe, journal et état du flux [f13-preuve.json] · Alias avant et après [f13-registre.txt] · Reproduction [f13-repro.txt]